

Exposé für die Projektarbeit

Entwicklung einer Custom-Engine für generative Spielwelten und Reinforcement Learning

Florian Merlau

Laurin [Nachname]

März 2026

1 Projektidee

Wir planen in unserer Projektarbeit die Entwicklung eines 2D-Top-Down-Spiels (im Stil von Minecraft), welches als experimentelle Testumgebung für Künstliche Intelligenz (Reinforcement Learning) konzipiert ist.

Im Gegensatz zu Projekten, die feste Level verwenden, soll unsere Architektur auf einer deterministischen, prozedural generierten „Chunk“-Welt basieren. Dieser Ansatz dient der Förderung der Generalisierungsfähigkeit des KI-Agenten [2, 5]. Dadurch variieren die Level bei jedem Durchlauf anhand eines Seeds. Der trainierte KI-Agent kann somit keine Maps auswendig lernen, sondern muss allgemeingültige Lösungsstrategien (Ressourcen abbauen, Hindernissen ausweichen, Zielpunkte finden) entwickeln.

2 Kurzes Glossar: Wichtige Fachbegriffe

Damit die Konzepte im restlichen Exposé nachvollziehbar bleiben, werden hier die zentralen Fachbegriffe kurz erklärt:

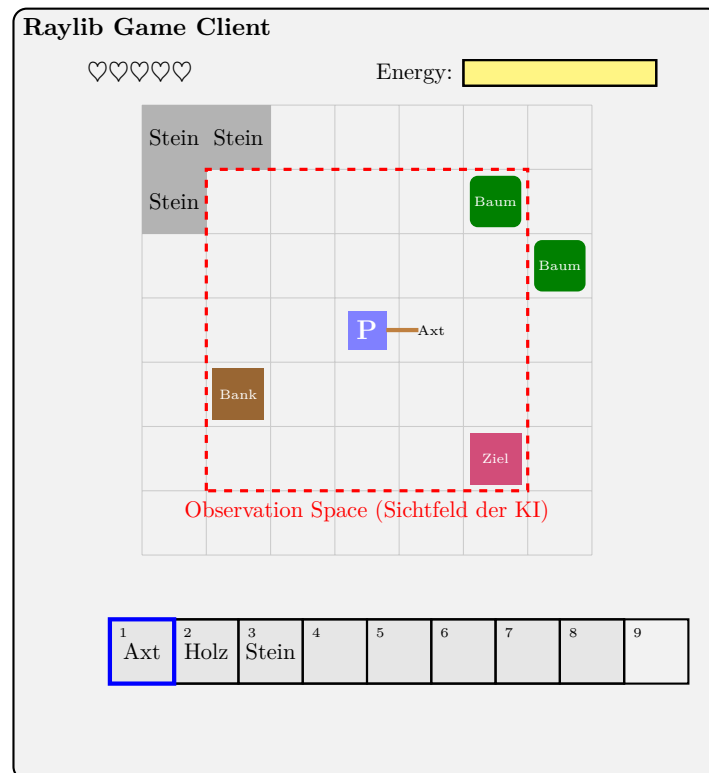
- **Reinforcement Learning (RL):** Eine Technik der Künstlichen Intelligenz, die durch Interaktion lernt. Für nützliche Aktionen gibt es eine Belohnung (Reward), für unvorteilhafte Aktionen nicht (vergleichbar mit Konditionierung).
- **Prozedurale Generierung & Seed:** Die Spielwelt wird nicht vorab modelliert, sondern bei jedem Start von einem Algorithmus generiert. Der *Seed* ist dabei der zufällige Startwert: Gibt man denselben Startwert ein, baut der Algorithmus exakt dieselbe Welt auf.
- **Observation Space:** Der Eingabevektor der KI. Die KI empfängt eine Tabelle aus Zahlen, die beschreibt, was in der lokalen Umgebung passiert und wie der innere Zustand ist.
- **Headless-Modus & Vektorisiert:** „Headless“ (kopflös) bedeutet, dass das Spiel ohne Grafikausgabe im Hintergrund simuliert wird, um Berechnungszeit zu sparen. „Vektorisiert“ heißt, dass das Spiel mehrfach instanziiert wird, sodass viele KI-Agenten parallel trainieren können.

- **Sparse vs. Dense Rewards:** „Sparse“ (selten) bedeutet, es gibt die Belohnung erst am Ende der Episode, wenn das Level gelöst wurde. „Dense“ (dicht) bedeutet, es werden auch Zwischenschritte (z.B. der Abbau von Bäumen) mit kleineren Belohnungen honoriert.

3 Methodische Visualisierung: Das Spielkonzept

Um die Schnittstelle zwischen der grafischen Darstellung (für menschliche Spieler) und den Rohdaten (für die KI) zu verdeutlichen, zeigt die folgende Skizze das Kernlayout der Engine (*StoneforgeFrontier*).

Während der visuelle Client (Raylib) die gesamte Karte sowie das UI (Leben, Energie, Inventar-Hotbar) rendert, erhält der KI-Agent als Eingabe (*Observation Space*) in erster Linie einen numerischen Array des lokalen Umfelds (hier als rote gestrichelte Box dargestellt). Um zielloses Umherirren in der prozeduralen Welt zu verhindern, erhält der Agent zusätzlich globale Statusinformationen in Form eines Richtungsvektors zum Ziel sowie den aktuellen Inventar- und Gesundheitsstatus. Das Handlungsmodell (*Action Space*) des Agenten wird dabei diskret auf ein Gitter-Modell abgebildet und umfasst elementare Bewegungs- sowie Interaktionsbefehle (Bauen, Abbauen, Benutzen).



4 Technologien und Architektur

Für das CPU-intensive KI-Training verzichten wir auf vorgefertigte Game-Engines (wie Unity) und entwickeln eine eigene Datenarchitektur:

4.1 Engine und Schnittstellen

- **C++20 (Core Engine):** Die Spielmechanik wird in C++ implementiert. Das ermöglicht einen „Headless“-Modus, durch den die KI das Rendern von Grafiken überspringen kann. Dies erlaubt es, mehrere Instanzen parallel auszuführen (*Vectorized Environments*), was den Trainingsprozess beschleunigt.
- **Python-Bindings (Pybind11 & Gymnasium):** Um die C++-Engine mit den KI-Algorithmen zu verbinden, programmieren wir Schnittstellen via *pybind11*. Das Spiel wird dadurch als standardisiertes *Gymnasium*-Environment in Python ladbar.

4.2 Prozedurale Weltengenerierung

Die Architektur der Welten benötigt eine Balance aus Varianz und Struktur (PCG-Taxonomie nach [7]). Dafür setzen wir auf einen hybriden Ansatz:

- **Layered Noise & Cellular Automata:** Die Basis für Biom-Struktur und Ressourcenverteilung bildet Rauschen (Noise). Um definierte Hindernisse und Höhlensysteme zu schaffen, glätten wir diese Struktur mit Zellulären Automaten.
- **Flood-Fill Validierung:** Wir implementieren einen Flood-Fill-Algorithmus, der sicherstellt, dass Zielpunkte physisch erreichbar bleiben. Ein unlösbarer Seed würde das KI-Training andernfalls blockieren.
- **Lazy Evaluation:** Die Engine generiert Chunks *on-demand* (lazy). Level-Bereiche werden berechnet, wenn die KI sich nähert, was den RAM-Bedarf reduziert. Um dabei zu verhindern, dass die KI in unlösbare Sackgassen läuft, stellt ein vorgelagerter Makro-Graph die Erreichbarkeit des Ziels auf globaler Ebene sicher, bevor die feingranularen Chunks instanziiert werden.

5 KI-Algorithmen und Training

Für das Training in den prozeduralen Umgebungen nutzen wir folgendes RL-Setup:

- **PPO (Proximal Policy Optimization) [6]:** Ein Policy-Gradient-Verfahren (via *Stable-Baselines3* [4]), das in der Praxis oft stabile Lernkurven aufweist.
- **Upgrades bei Stagnation:** Sollte der Agent im Grid stagnieren, planen wir funktionelle Erweiterungen wie **PPO-LSTM** (Kurzzeitgedächtnis für Navigation in teilweise beobachtbaren Umgebungen) oder **Intrinsic Curiosity / RND** [3, 1] zur Motivationssteuerung.

5.1 Reward Design und Evaluierung

Das Design der Belohnungsfunktion (Reward Function) hat Einfluss auf das Verhalten des Agenten. Um stark verzögerten Belohnungen (*Sparse Rewards*) entgegenzuwirken, setzen wir auf ein hybrides Modell: Der Agent erhält einen *Sparse Reward* für das Erreichen des Ziels, gestützt durch kleinere *Dense Rewards* für progressionsfördernde Aktionen (z.B. den Abbau benötigter Ressourcen oder die Reduzierung der Distanz). Durch gezieltes *Reward Shaping* – etwa durch zeitliche Abstrafung oder Begrenzung der Belohnung für

Rohstoffsammlung – wird dabei sichergestellt, dass der Agent eine zielführende Strategie entwickelt und Fehlverhalten (z.B. reines Ressourcen-Sammeln ohne Exploration des Ziels) vermieden wird.

Als Evaluierungsmetrik definieren wir eine **Success Rate** von $\geq 90\%$ über ein Set von 100 *unbekannten* Seeds. Des Weiteren betrachten wir die Länge der gefundenen Pfade anhand der Minimierung der *Steps per Episode*.

6 Projektziele

6.1 Must-Have-Kriterien

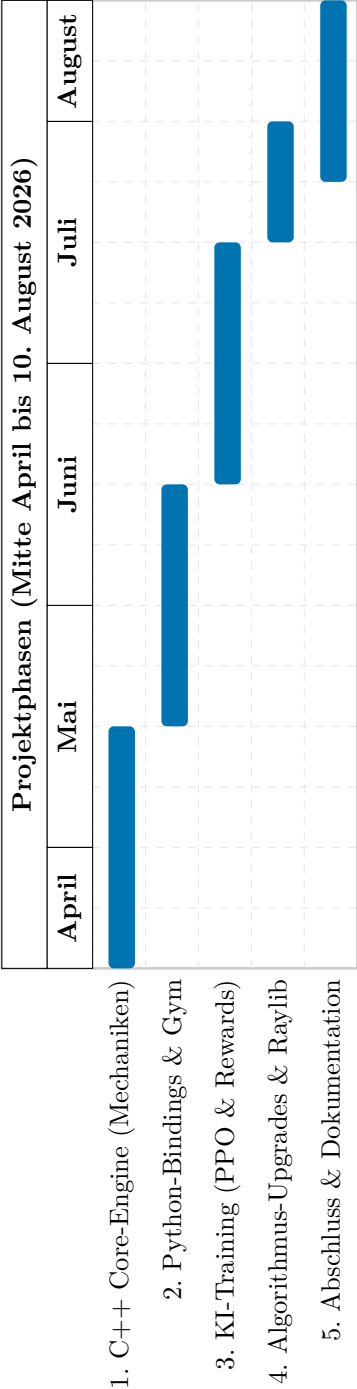
1. Entwicklung der C++-Engine mit deterministischer Chunk-Generierung.
2. Brücke zwischen C++ und Python mittels *pybind11* als vektorisiertes *Gymnasium*-Environment.
3. Basis-Training eines PPO-Agenten, der auf unbekannten Seeds sein Zielgebiet erreicht.

6.2 Nice-to-Have-Kriterien

1. Grafischer Raylib-Client zur Live-Beobachtung (wie in der Skizze zu sehen).
2. System-Upgrades wie PPO-LSTM oder Curiosity (RND/ICM).
3. Erweitertes Crafting-System (Holz abbauen als Voraussetzung für Werkzeuge).

7 Zeitplan

Die Phase der Umsetzung erstreckt sich über 4,5 Monate (Mitte April bis 10. August). Der Zeitplan ist wie folgt aufgeteilt:



Literatur

- [1] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations*, 2018. <https://arxiv.org/abs/1810.12894>.
- [2] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning*, pages 2048–2056. PMLR, 2020. <https://arxiv.org/abs/1912.01588>.
- [3] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017. <https://arxiv.org/abs/1705.05363>.
- [4] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. <http://jmlr.org/papers/v22/20-1364.html>.
- [5] Sebastian Risi and Julian Togelius. Increasing generality in machine learning through procedural content generation. *Nature Machine Intelligence*, 2(8):428–436, 2020. <https://www.nature.com/articles/s42256-020-0208-5>.
- [6] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>.
- [7] Julian Togelius, Georgios Kastadjian, Mohammed S Faust, and Georgios N Yannakakis. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011. <https://ieeexplore.ieee.org/document/5963131>.